Draw the standard Process State Transition Diagram.
Then analyze what will happen if the scheduler is not able to move processes from *Ready* → *Running* efficiently.
Explain its impact on CPU utilization.

Differentiate between Ready, Running, and Blocked states.
Provide a scenario where a process repeatedly transitions between *Running* → *Blocked* → *Ready*.
Analyze why such behaviour may reduce system throughput.

Then analyze a situation where a large number of zombies accumulate in the system.
What would be the impact on system resources?

A process makes a blocking I/O call and moves to the Waiting state.
Analyze the conditions under which this process might suffer from indefinite waiting.
Suggest at least one OS-level mitigation.

A process remains in the Blocked/Waiting state for a long time.
Analyze at least two root causes, one related to resource allocation and the other related to scheduling policy.

Describe the key components of a Process Control Block (PCB).
Then analyze the role of PCB during context switching and how it affects OS performance.

A ready process is repeatedly denied CPU time for multiple scheduling cycles.
Analyze this scenario in the context of starvation and CPU fairness policies.

FCFS Scheduling

Given:

Process | AT | BT
P1 | 0 | 12
P2 | 3 | 5
P3 | 5 | 8

(A) Draw the Gantt chart and calculate WT & TAT for all processes.
(B) ANALYZE: Explain how the arrival pattern affects FCFS performance. Would the result improve if P2 arrived earlier?

2. Non-Preemptive SJF

Process | AT | BT
P1 | 0 | 7
P2 | 2 | 3
P3 | 4 | 1
P4 | 5 | 5

(A) Draw the Gantt chart, WT, TAT, and Avg WT.
(B) ANALYZE: Evaluate how SJF minimizes waiting time. Identify which process benefits the most from SJF and why.

3. Preemptive SJF (SRTF)

Process | AT | BT
P1 | 1 | 8
P2 | 2 | 4
P3 | 3 | 2
P4 | 5 | 1

(A) Construct Gantt chart and compute WT, TAT.
(B) ANALYZE: Explain how preemption changes the execution order compared to non-preemptive SJF.

4. Round Robin (TQ = 3 ms)

Process | AT | BT
P1 | 0 | 10
P2 | 1 | 4
P3 | 2 | 6

(A) Draw the Gantt chart, CT, WT, TAT, RT.
(B) ANALYZE: Show how reducing the quantum to 2 ms would increase context switching. Would overall performance improve?

5. Round Robin (TQ = 5 ms)

Process | AT | BT
P1 | 0 | 18
P2 | 2 | 7
P3 | 4 | 12

(A) Create the Gantt chart and compute all metrics.
(B) ANALYZE: Discuss whether a quantum of 5 ms is optimal here. What happens if the quantum becomes larger than the largest BT?

6. Priority Scheduling (Non-Preemptive)

Lower number = higher priority
Process | Pri | BT
P1 | 3 | 6
P2 | 1 | 5
P3 | 4 | 2
P4 | 2 | 8

(A) Compute WT and TAT.
(B) ANALYZE: Identify if any process suffers from starvation. How does changing priorities dynamically solve this?

Dining Philosophers – 10 Questions

1. Philosopher picks left chopstick and waits forever for right—identify the issue.
2. All philosophers pick left chopstick simultaneously—what problem occurs?
3. One philosopher eats much more often—identify synchronization flaw.
4. Philosopher releases one chopstick early—describe the effect.
5. Increasing philosophers from 5 to 7—impact on deadlock risk.
6. Circular wait among all—identify responsible deadlock condition.
7. Incorrect ordering of semaphores leads to deadlock—explain how.
8. Waiter solution added—what problem does it eliminate?
9. Rapid switching between eating and thinking increases contention—explain why.
10. Slow philosopher holds chopsticks too long—what issue arises?

Reader–Writer – 10 Questions

11. Readers enter frequently; writers wait too long—identify the issue.
12. Readers allowed during writer's critical section—what synchronization failed?
13. Readers cause writer starvation—suggest a fix.
14. Writer finishes but doesn't signal readers—what is the consequence?
15. Priority to writers—why does reader throughput drop?
16. Implementing fairness—how does it change access?
17. Reader crashes inside critical section—effect on the system.
18. Writer interrupted by readers mid-update—analyze this condition.
19. Incorrect lock implementation allows multiple writers—identify the flaw.
20. Log writers not prioritized—impact on logging system.

Producer–Consumer – 10 Questions

- Producer faster than consumer—what problem occurs?
- Consumer tries to read empty buffer—what prevents this?
- Buffer permanently full—what deadlock condition caused this?
- Producer fails to signal consumer after producing—what is the result?
- Consumer reads wrong buffer index—explain the race condition.
- Two producers write at the same time without lock—effect.
- Different semaphore order—risk of deadlock—explain.
- Very large buffer used—performance implications.
- Slow consumer affects queue stability—how?
- Two consumers retrieve the same item—explain what happens.

Sleeping Barber Problem – 10 Questions

- Barber sleeps but no customer wakes him—what synchronization mistake occurred?
- Waiting room has N chairs; customers exceed capacity—what must happen to extra customers?
- Barber finishes haircut but fails to signal next customer—what is the effect?
- Multiple customers arrive at the same time—how should access to waiting chairs be controlled?
- Customer gets into barber chair without waking barber—what synchronization primitive failed?
  Barber starts cutting two customers at once—identify locking issue.
  Semaphore allows multiple customers into haircut area though only one barber exists—explain flaw.
  Customer finds barber busy but sits in barber chair—describe this race condition.
  Barber is idle but customers still wait in queue—what scheduling or signal order is wrong?Waiting room is empty; barber finishes haircut—explain correct transition to sleeping state.

Deadlock Handling Methods

1. Explain the three primary strategies to handle deadlock:
   Deadlock Prevention ,Deadlock Avoidance and Deadlock Detection and Recovery
2. Compare their advantages and disadvantages.
3. Provide examples of algorithms or OS mechanisms for each strategy (e.g., Banker's Algorithm for avoidance).

Deadlock Questions – Problem Solving similar to gate exam questions

File Management Questions

A. Theory/Conceptual Questions (5–6 Questions)

1. Define a file and explain the difference between a file and a record.
2. What are the various types of files in an operating system (e.g., text, binary, executable)? Give examples.
3. Explain the different file operations supported by an OS (create, read, write, delete, rename, append).
4. Compare sequential, direct (random), and indexed access files with examples.
5. Define file attributes. Explain why they are important for file management.
6. Explain the differences between primary memory storage and secondary memory storage in terms of file handling.

B. File Organization & Allocation (5–6 Questions)

7. Describe contiguous, linked, and indexed file allocation methods. Discuss advantages and disadvantages of each.
8. A disk has 1000 blocks. File F1 requires 120 blocks and file F2 requires 300 blocks. If contiguous allocation is used, explain how external fragmentation might occur.
9. Explain the concept of inode-based file allocation and its importance in UNIX/Linux file systems.
10. Compare sequential access vs direct access in terms of file allocation efficiency.
11. Describe free space management techniques: bit vector, linked list, and grouping.
12. Explain how indexing with multiple levels helps in efficient file access.

C. Directory Structure & File System Questions (4–5 Questions)

13. Explain the difference between single-level, two-level, tree-structured, and acyclic-graph directory structures.
14. A system implements a tree-structured directory. Explain how absolute and relative paths are used to locate a file.
15. Discuss file protection mechanisms and how access rights (read/write/execute) are implemented.
16. Explain the role of the File Control Block (FCB) in managing files.
17. How do symbolic links and hard links differ in UNIX/Linux file systems?

Memory Management Questions

A. Theory / Conceptual Questions

1. Define memory management and explain its importance in operating systems.
2. Differentiate between contiguous and non-contiguous memory allocation.

3. What is paging? Explain logical vs physical addresses.
4. Define demand paging and explain how it differs from simple paging.
5. Explain page faults. What happens when a page fault occurs?
   Discuss the advantages and disadvantages of virtual memory.

C. Problem-Solving / Numerical Questions

Problem 1: FIFO Page Replacement

- Given a reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2
- Number of frames = 3
  Tasks:

1. Draw the FIFO page replacement table.
2. Calculate the total number of page faults.

Problem 2: LRU Page Replacement

- Reference string: 1, 2, 3, 4, 2, 1, 5, 2, 4, 5
- Number of frames = 3
  Tasks:

1. Simulate LRU page replacement.
2. Calculate page faults.

Problem 3: Optimal Page Replacement

- Reference string: 2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2
- Number of frames = 3
  Tasks:

1. Simulate optimal page replacement.
2. Count page faults.

Problem 4: Demand Paging Scenario

- A process has 5 pages and 3 frames. Reference string: 0, 2, 1, 3, 0, 4, 2, 3, 0
- Using FIFO page replacement, determine page faults.
- Explain what happens at each page fault in demand paging.

Problem 5: Belady's Anomaly

- Show an example of FIFO page replacement where increasing the number of frames increases page faults.
- Explain why it occurs.

Problem 6: Working Set Calculation

- Given a reference string and window size 4, identify the working set of the process at each step.
- Explain how this reduces thrashing.

Problem 7: LFU Page Replacement

- Reference string: 3, 2, 1, 4, 3, 2, 5, 2, 3, 2, 4
- Number of frames = 3
  Task: Simulate LFU page replacement and calculate page faults.